Day 2 – Lab 2.2: Blob → Queue → Blob Processing

Azure Functions · C# · .NET 10 Isolated Worker · Azurite

Objective: Build a local event-driven order-processing workflow. When `orders.csv` is uploaded to Blob Storage, a Blob-triggered Function validates each record and sends valid orders to Queue Storage. A second Function consumes the messages and writes individual JSON files to Blob Storage.

Business scenario: Contoso Orders receives order files from an external system. Instead of processing the entire file in one operation, the application separates file ingestion from individual order processing using a queue.

Important: This lab runs locally using Azurite. No Azure resources are created or modified. Keep the Day 2 Azure Function App and Storage Account for Lab 2.3.

1. Prerequisites

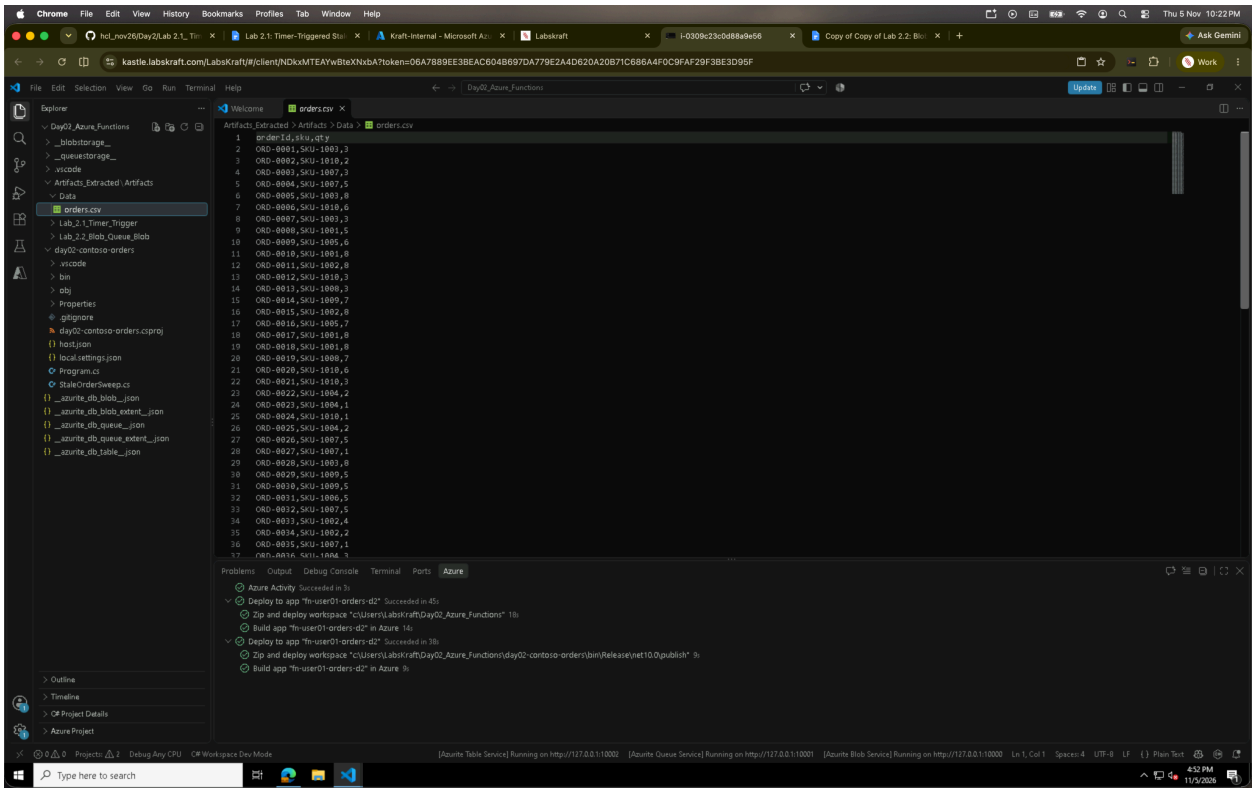
Complete Lab 2.1 and confirm:

- Existing project: `contoso-fn`
- .NET 10 SDK installed
- Azure Functions Core Tools 4.x
- VS Code with Azure Functions and Azurite extensions
- Existing training dataset: `orders.csv`

Expected CSV columns:

```
None
orderId,sku,qty
```

Use the supplied training asset; do not create a replacement dataset.



2. Install the Storage extensions

Open the existing project in VS Code:

```
None
cd contoso-fn
```

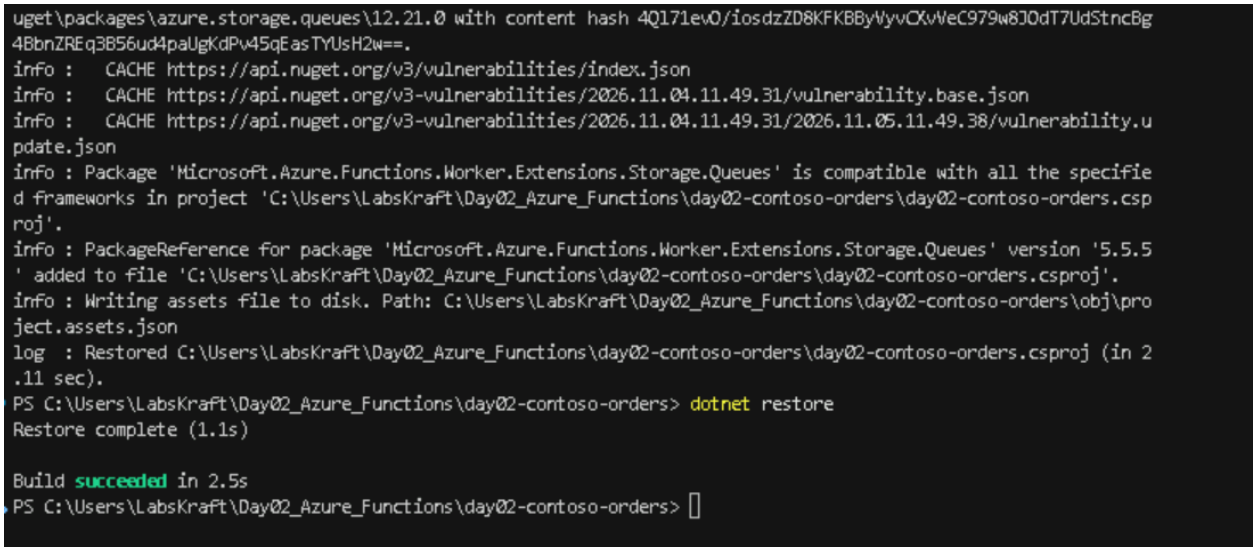
Install the required packages:

```
None
dotnet add package
Microsoft.Azure.Functions.Worker.Extensions.Storage.B
lobs
dotnet add package
Microsoft.Azure.Functions.Worker.Extensions.Storage.Q
ueues
dotnet restore
```

These extensions provide Blob and Queue triggers and bindings for the .NET isolated worker.

```
PS C:\Users\LabsKraft\Day02_Azure_Functions> cd day02-contoso-orders
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders> dotnet add package Microsoft.Azure.Functions.
Worker.Extensions.Storage.Blobs
  info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
  info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
  info : Adding PackageReference for package 'Microsoft.Azure.Functions.Worker.Extensions.Storage.Blobs' into pro
  ject 'C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj'.
  info :   GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.azure.functions.worker.extensions.stor
  age.blobs/index.json
  info :   OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.azure.functions.worker.extensions.stora
  ge.blobs/index.json 203ms
  info : Restoring packages for C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-order
  s.csproj...
  info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.storage.blobs/i
  ndex.json
  info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.storage.blobs/in
  dex.json 210ms
  info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.storage.blobs/6
  .8.2/microsoft.azure.functions.worker.extensions.storage.blobs.6.8.2.nupkg
  info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.storage.blobs/6.
  8.2/microsoft.azure.functions.worker.extensions.storage.blobs.6.8.2.nupkg 90ms
  info :   GET https://api.nuget.org/v3-flatcontainer/azure.storage.blobs/index.json
  info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/index.json
  info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.azure/index.json
  info :   OK https://api.nuget.org/v3-flatcontainer/azure.storage.blobs/index.json 419ms
  info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.core/index.json 437ms
```

```
Worker.Extensions.Storage.Blobs
  added to file 'C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj'.
info : Generating MSBuild file C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\obj\day02-contoso-
orders.csproj.nuget.g.targets.
info : Writing assets file to disk. Path: C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\obj\pro
ject.assets.json
log  : Restored C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj (in 6
.16 sec).
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders> dotnet add package Microsoft.Azure.Functions.
Worker.Extensions.Storage.Queues
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
info : Adding PackageReference for package 'Microsoft.Azure.Functions.Worker.Extensions.Storage.Queues' into pr
oject 'C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-orders.csproj'.
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.azure.functions.worker.extensions.stor
age.queues/index.json
info :   OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.azure.functions.worker.extensions.stora
ge.queues/index.json 520ms
info : Restoring packages for C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\day02-contoso-order
s.csproj...
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.storage.queues/
index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.storage.queues/i
ndex.json 277ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.azure.functions.worker.extensions.storage.queues/
```



3. Create the Blob-triggered Function

Create a file named `SplitCsv.cs` in the project.

Add:

```
None
using System.Text.Json;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;

namespace ContosoFn;

public class SplitCsv
{
    private readonly ILogger<SplitCsv> _logger;

    public SplitCsv(ILogger<SplitCsv> logger)
    {
```

```
        _logger = logger;
    }

    [Function("SplitCsv")]
    [QueueOutput("orders", Connection =
"AzureWebJobsStorage")]
    public string[] Run(
        [BlobTrigger(
            "incoming/{name}",
            Connection = "AzureWebJobsStorage")]
        Stream blob,
        string name)
    {
        _logger.LogInformation(
            "Processing uploaded file: {FileName}",
name);

        using var reader = new StreamReader(blob);

        var lines = reader.ReadToEnd()
            .Split('\n',
StringSplitOptions.RemoveEmptyEntries);

        if (lines.Length < 2)
        {
            _logger.LogWarning("CSV contains no order
records.");
```

```
            return Array.Empty<string>();
        }

        var headers = lines[0].Trim()
            .Split(',')
            .Select(h => h.Trim())
            .ToArray();

        int orderIdIndex = Array.IndexOf(headers,
"orderId");
        int skuIndex = Array.IndexOf(headers, "sku");
        int qtyIndex = Array.IndexOf(headers, "qty");

        if (orderIdIndex < 0 || skuIndex < 0 ||
qtyIndex < 0)
        {
            _logger.LogError(
                "Required CSV columns are missing:
orderId, sku, qty.");

            return Array.Empty<string>();
        }

        var messages = new List<string>();

        foreach (var line in lines.Skip(1))
        {
```

```
            var fields = line.Trim().Split(',');

            if (fields.Length != headers.Length)
                continue;

            var orderId =
fields[orderIdIndex].Trim();
            var sku = fields[skuIndex].Trim();
            var qtyText = fields[qtyIndex].Trim();

            if (string.IsNullOrWhiteSpace(orderId) ||
                string.IsNullOrWhiteSpace(sku) ||
                !int.TryParse(qtyText, out int qty)
||
                qty <= 0)
            {
                _logger.LogWarning(
                    "Skipping invalid order record:
{Record}", line);

                continue;
            }

            var message =
JsonSerializer.Serialize(new
            {
                orderId,
```

```
                sku,
                qty
            });

            messages.Add(message);
        }

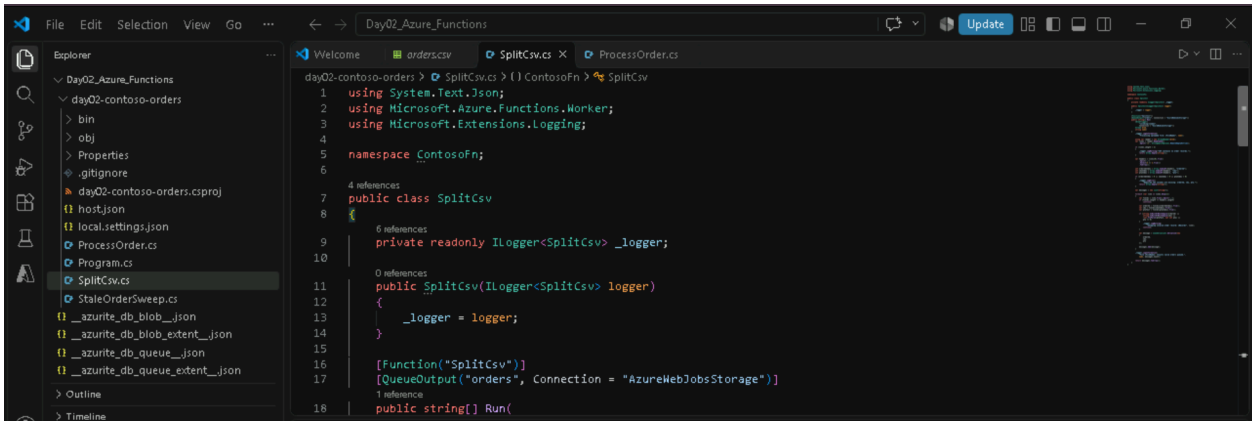
        _logger.LogInformation(
            "File {FileName}: {Count} valid orders
queued.",
            name, messages.Count);

        return messages.ToArray();
    }
}
```

Function behaviour:

- Monitors the `incoming` Blob container.
- Executes when a Blob is uploaded.
- Reads the CSV file.
- Validates `orderId`, `sku` and positive `qty`.
- Creates one JSON queue message for each valid order.
- Sends all valid messages to the `orders` queue.

The code uses simple comma-separated parsing, appropriate for the supplied training dataset. It is not intended to parse CSV fields containing embedded commas or quoted multiline values.



4. Create the Queue-triggered Function

Create another file:

`ProcessOrder.cs`

Add:

```
None
using System.Text.Json;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;

namespace ContosoFn;

public class ProcessOrder
{
    private readonly ILogger<ProcessOrder> _logger;

    public ProcessOrder(ILogger<ProcessOrder> logger)
    {
        _logger = logger;
```

```
    }

    [Function("ProcessOrder")]
    [BlobOutput(
        "processed/{rand-guid}.json",
        Connection = "AzureWebJobsStorage")]
    public string Run(
        [QueueTrigger(
            "orders",
            Connection = "AzureWebJobsStorage")]
        string message)
    {
        _logger.LogInformation(
            "Received queue message: {Message}",
message);

        using var document =
JsonDocument.Parse(message);

        var orderId = document.RootElement
            .GetProperty("orderId").GetString();

        var sku = document.RootElement
            .GetProperty("sku").GetString();

        var qty = document.RootElement
            .GetProperty("qty").GetInt32();
```

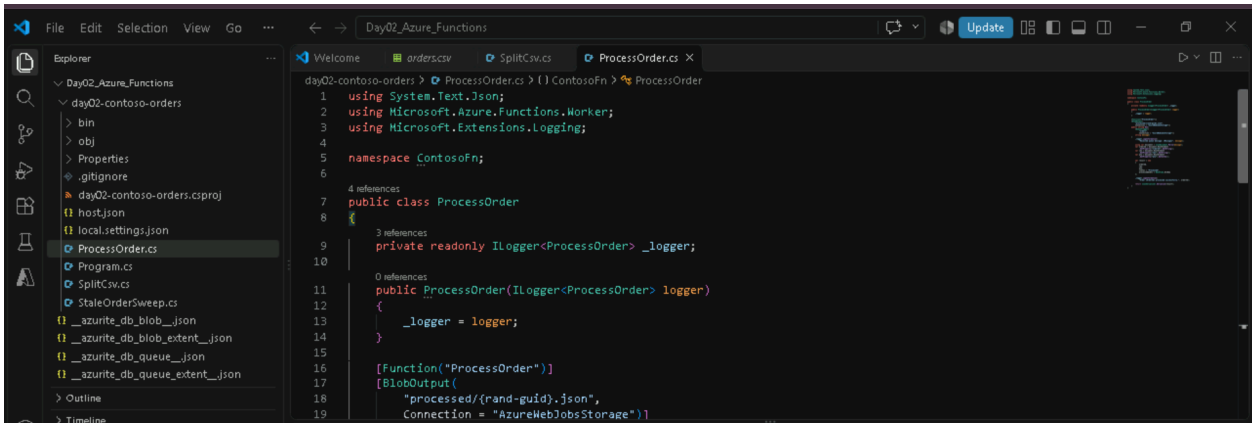
```
        var result = new
        {
            orderId,
            sku,
            qty,
            status = "Processed",
            processedAtUtc = DateTime.UtcNow
        };

        _logger.LogInformation(
            "Order {OrderId} processed
successfully.", orderId);

        return JsonSerializer.Serialize(result);
    }
}
```

Function behaviour:

- Listens to the `orders` queue.
- Receives one order message.
- Reads the order details.
- Creates a processed JSON record.
- Writes the JSON file into `processed`.
- Uses `{rand-guid}` to generate a unique Blob name.



5. Prepare local storage using Azurite

Start Azurite:

1. Open VS Code.
2. Press `Ctrl + Shift + P`.
3. Select `Azurite: Start`.

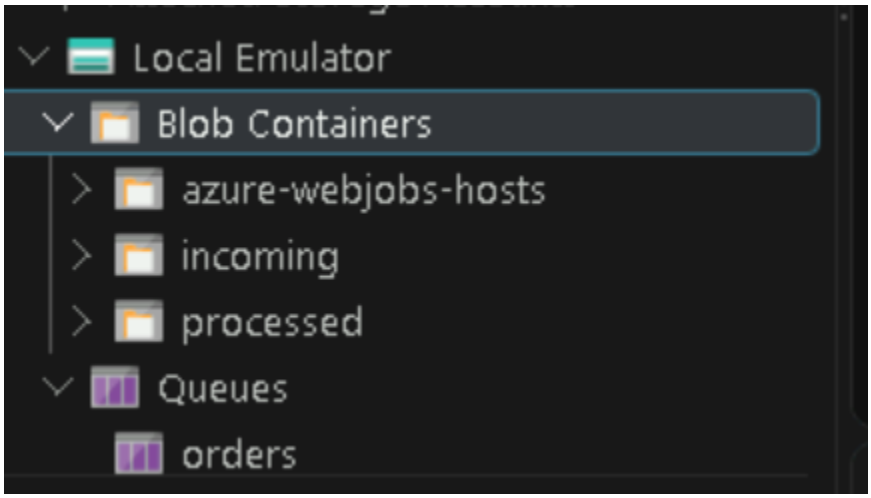
Open Azure Storage Explorer.

1. Connect to Emulator – Default Ports.
2. Expand the local emulator storage account.
3. Open Blob Containers.
4. Create:

```
None
incoming
processed
```

5. Open Queues.
6. Create:

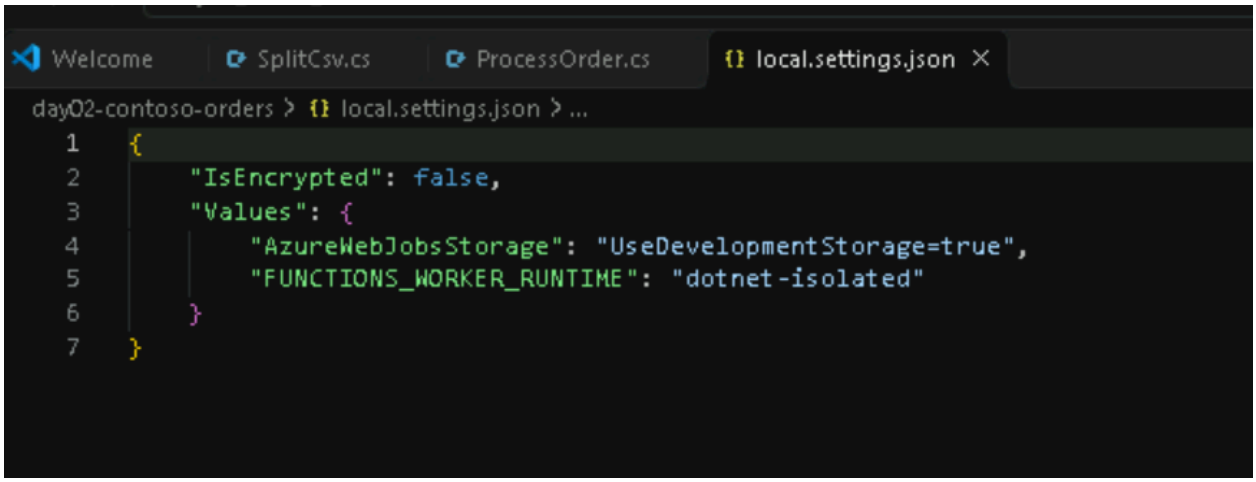
```
None
orders
```



Verify that your existing `local.settings.json` contains:

```
None
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage":
"UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated"
  }
}
```

If the file already contains additional settings, retain them and update only the required values.



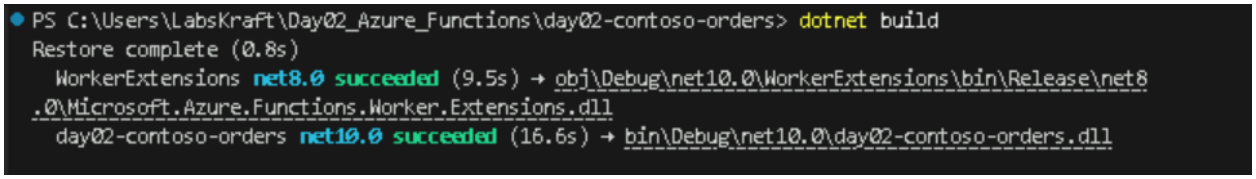
6. Build and run the Functions project

Run:

```
None
dotnet build
```

Expected:

```
None
Build succeeded.
0 Error(s)
```



Start the local Functions host:

```
None
func start
```

Confirm these Functions are listed:

```
None
QuoteFunction
StaleOrderSweep
SplitCsv
ProcessOrder
```

The first two are retained from previous labs.

```
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders> func start
Running 'func start' directly against a .NET Isolated project may not correctly load function ex
tensions. Use 'dotnet run' instead, which builds the project and starts the Functions host from
the correct output directory.
  Determining projects to restore...
  All projects are up-to-date for restore.
  Determining projects to restore...
  All projects are up-to-date for restore.
  WorkerExtensions -> C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\obj\Debug\ne
t10.0\WorkerExtensions\bin\Release\net8.0\Microsoft.Azure.Functions.Worker.Extensions.dll
  day02-contoso-orders -> C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\bin\outp
ut\day02-contoso-orders.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:07.89
```

```
[2026-11-05T17:25:49.030Z] Found C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\d
ay02-contoso-orders.csproj. Using for user secrets file configuration.
[2026-11-05T17:25:55.688Z] Worker process started and initialized.

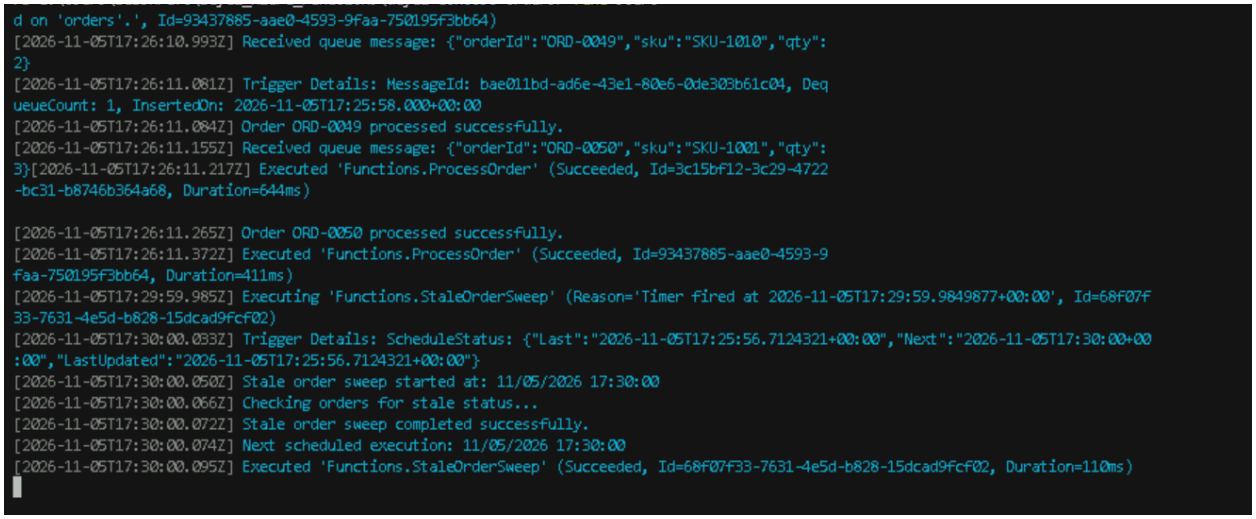
Functions:

        ProcessOrder: queueTrigger

        SplitCsv: blobTrigger

        StaleOrderSweep: timerTrigger

For detailed output, run func with --verbose flag.
[2026-11-05T17:25:56.766Z] Executing 'Functions.StaleOrderSweep' (Reason='Timer fired at 2026-11
-05T17:25:56.7283304+00:00', Id=4d560ef2-b8c8-4630-9f16-72445e4d29f7)
[2026-11-05T17:25:56.769Z] Trigger Details: UnscheduledInvocationReason: IsPastDue, OriginalSche
dule: 2026-11-05T15:15:00.0000000+00:00, ScheduleStatus: {"Last":"2026-11-05T15:10:00+00:00","Ne
xt":"2026-11-05T15:15:00+00:00","LastUpdated":"2026-11-05T15:10:00+00:00"}
[2026-11-05T17:25:57.227Z] Stale order sweep started at: 11/05/2026 17:25:57
[2026-11-05T17:25:57.321Z] Stale order sweep is running later than scheduled.
[2026-11-05T17:25:57.331Z] Checking orders for stale status...
[2026-11-05T17:25:57.334Z] Stale order sweep completed successfully.[2026-11-05T17:25:57.245Z] E
xecuting 'Functions.SplitCsv' (Reason='New blob detected(LogsAndContainerScan): incoming/"C:/Use
```



7. Execute the workflow

1. Open Storage Explorer.
2. Navigate to the local emulator → Blob Containers → `incoming`.
3. Select Upload → Upload Files.
4. Select the existing `orders.csv` training asset.
5. Upload the file.
6. Return to the VS Code terminal.
7. Observe the `SplitCsv` and `ProcessOrder` execution logs.

Expected processing sequence:

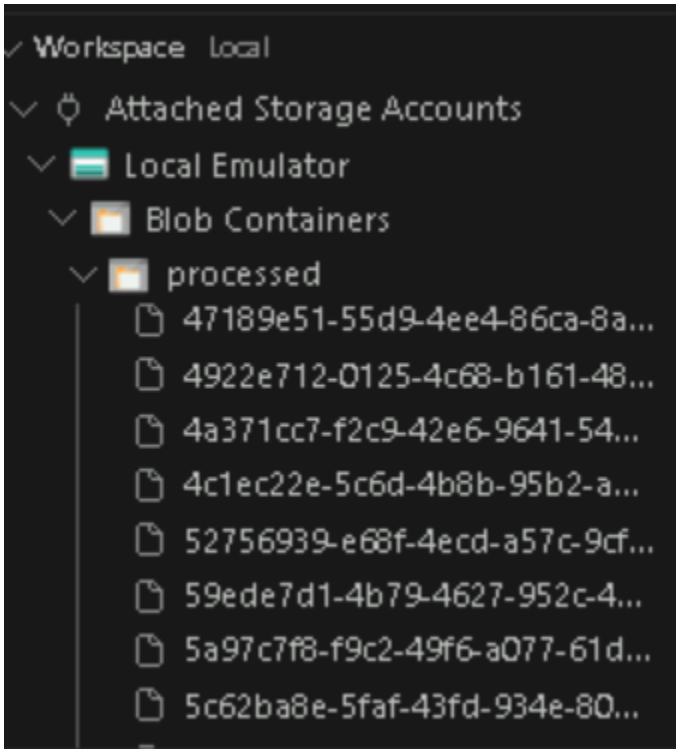
orders.csv

10 order records

SplitCsv

Reads and validates the records

orders queue



10

One message per valid order

ProcessOrder

Consumes and processes each message

processed container

10 JSON files

8. Validate the output

In Storage Explorer:

1. Open the `processed` Blob container.
2. Refresh the container.
3. Count the generated JSON files.
4. Open a JSON file and inspect its contents.

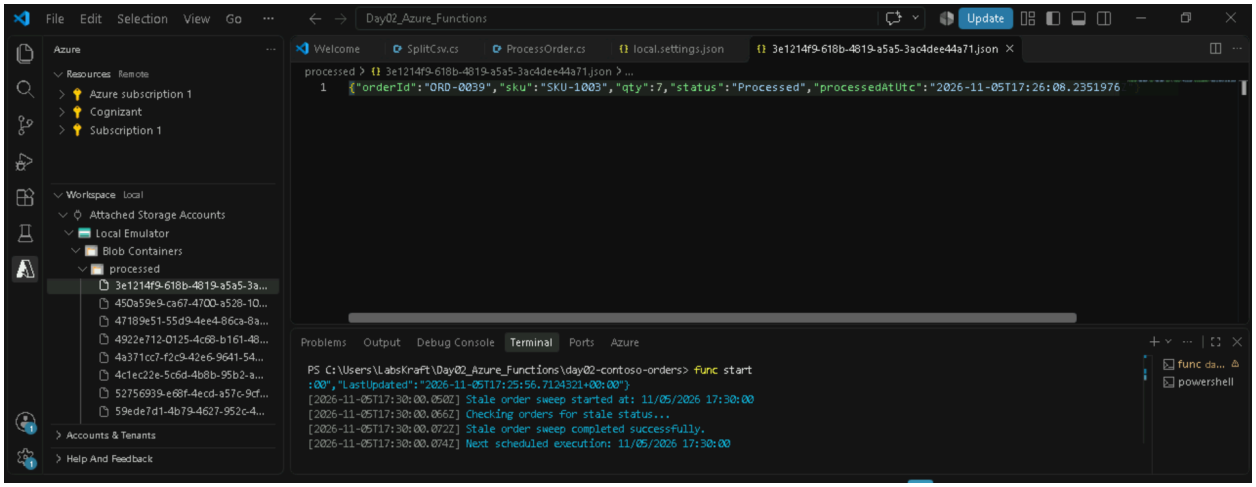
Expected output structure:

```
None
{
  "orderId": "ORD1001",
  "sku": "SKU101",
  "qty": 2,
  "status": "Processed",
  "processedAtUtc": "2026-11-04T09:30:00Z"
}
```

The values above illustrate the structure; actual order details come from the supplied CSV.

Expected result: Uploading the supplied 10-row CSV produces exactly 10 JSON files, each containing a valid `orderId`.

Note: Queue messages may be consumed immediately. Therefore, the queue may appear empty when you inspect it after processing. Use Function logs and the output Blob count to verify the workflow.



9. Troubleshooting

Issue	Check
Blob trigger does not execute	Azurite is running; upload into `incoming`
Function not listed	Storage extensions installed; build succeeds
Queue remains empty	CSV headers and validation logs
Queue trigger does not execute	Queue name is `orders`; Queue extension installed

JSON files not generated	Check `ProcessOrder` logs and `processed` container
Storage connection error	Verify `UseDevelopmentStorage=true` locally
Fewer than 10 files	Check CSV row count and skipped-record warnings

10. Acceptance criteria

Validation	Expected result
Azurite	Running
Containers	`incoming`, `processed`
Queue	`orders`
Blob trigger	`SplitCsv` executes

CSV processing	10 valid records
Queue output	10 order messages generated
Queue trigger	`ProcessOrder` executes
Blob output	Exactly 10 JSON files
JSON validation	Every file contains a valid `orderId`

11. Cleanup checkpoint

Local lab – no Azure cleanup required

- Stop the local Functions host using `Ctrl + C`.
- Stop Azurite if no longer needed.
- Preserve the C# project and dataset.
- Do not delete the Day 2 Azure Function App or Storage Account; Lab 2.3 needs them.

Production understanding: This architecture separates ingestion from processing. Blob Storage handles incoming files, the first Function converts records into independent queue messages, and the second

Function processes each order independently. In production, this pattern helps absorb bursts and isolate downstream processing failures.

Lab 2.2 complete.

Next: Lab 2.3 – Deploy the same Blob → Queue → Blob workflow to Azure Flex Consumption and configure Event Grid-based Blob triggering.